L7 · MESA AI-NATIVE TRACK · COHORT C4

You know how to build.

Now learn to own it.

L7 · MESA AI-Native Track · Cohort C4

Mesa
SCHOOL OF BUSINESS



This term, you shipped.

Connected APIs

Built automations and agents

Debugged

Deployed on Vercel

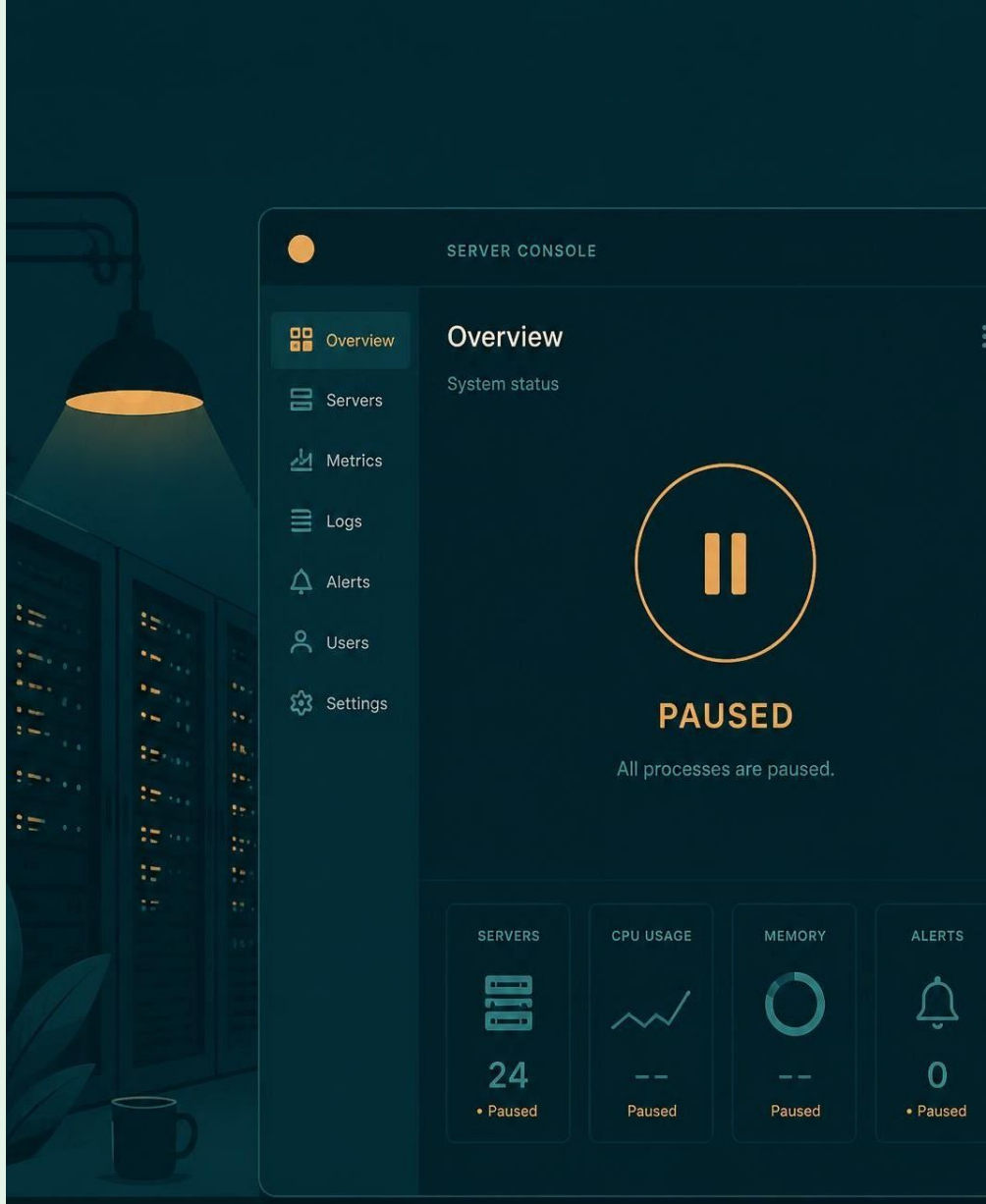
Shipped working applications

That is not a small thing. Most people never do it.

So - what's next?

The Art of Learning to Figure It Out.

Do you remember when Supabase paused your project?



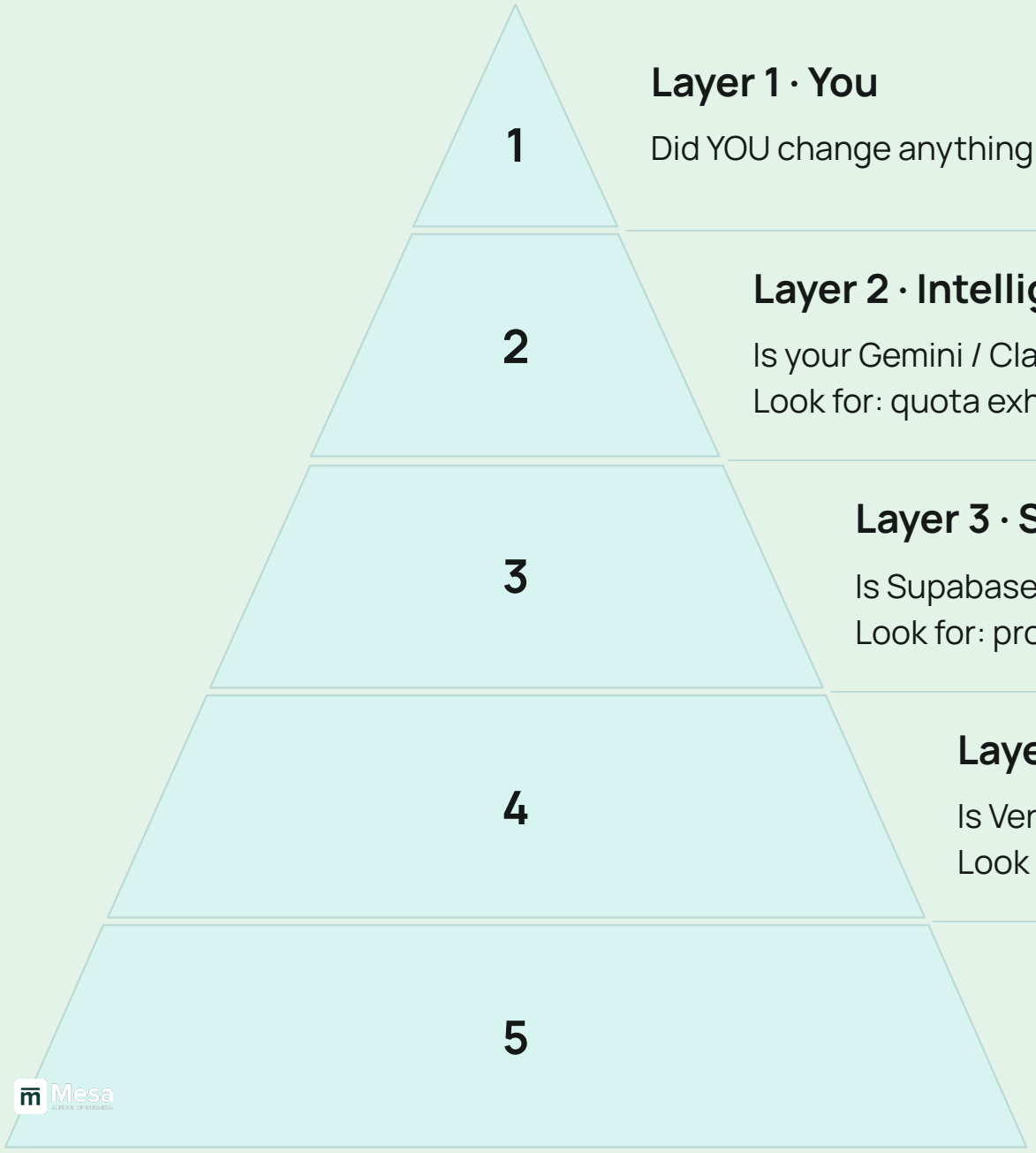
Diagnosis is the hard part. The fix is easy.

Right now: something breaks → you look in 100 places.

With experience: you rule out 80 immediately. You focus on the likely ones.

Once you've diagnosed it - you know exactly what to ask Claude.

You give it the right context, the right error, the right layer. It gives you the answer in fewer responses.



Layer 1 · You

Did YOU change anything in the last 24 hours? → Yes: start here → No: move down

Layer 2 · Intelligence (AI Provider)

Is your Gemini / Claude API healthy?
Look for: quota exhausted · key rotated · provider outage

Layer 3 · Storage (Database)

Is Supabase responding?
Look for: project paused · connection string broken · storage limit hit

Layer 4 · Delivery (Hosting)

Is Vercel serving your latest build?
Look for: deploy failed · env variable missing · build error

Layer 5 · Connections (External APIs)

Are your integrations alive?
Look for: token expired · webhook broken · third-party quota

How many of you built your own LinkedIn Post Generator?

How frequently do you use it?

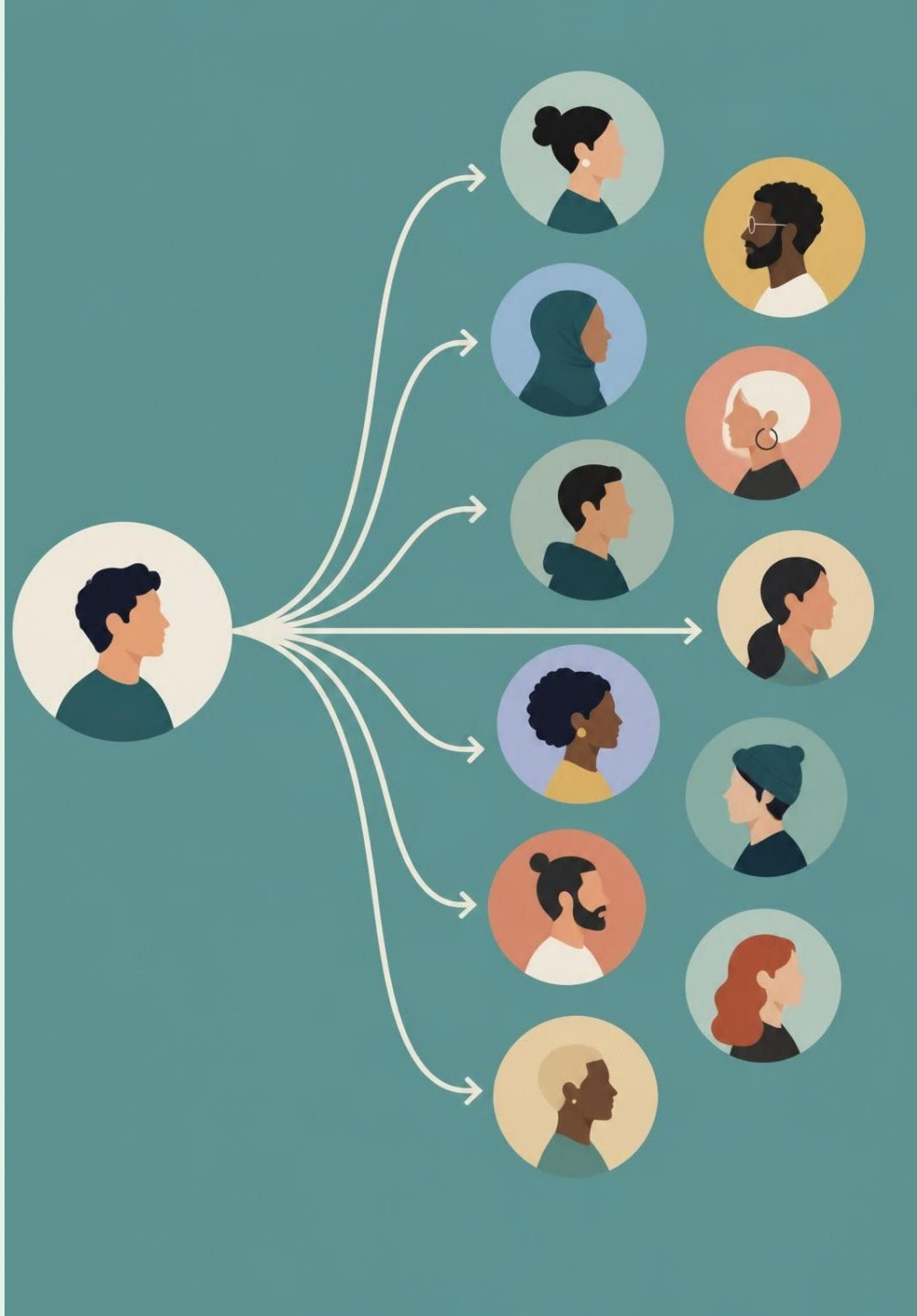
Did any of you try selling it?



What does it take to Scale?

When you built the LinkedIn bot, every decision you made was invisible — because you were the user. The moment someone else opens it, your assumptions become their problems.

When YOU use it	When a STRANGER uses it
You know it writes in your voice	They get posts that sound like the wrong person
You know to re-enter the key if it fails	They think the app is broken and stop using it
You know to refresh the page when it hangs	They write a bad review



Going from 1 to 5 lakh users

Rung	Who	What actually happens	What you need to add
1	You	Nothing breaks. You know every quirk. The app runs on tribal knowledge - yours.	Nothing yet.
5	Your batchmates	Priya's posts sound nothing like her. Rohan's API key expires and he has no idea where to get a new one. Every assumption you baked in is now their confusion.	Onboarding · error messages · a way to reach you
500	Paying customers	You're charging ₹999/month but the API bill hits ₹45,000. Three customers say the tone is wrong. You have no logs, no idea which output failed, no way to fix one person without breaking another.	Cost controls · per-user prompts · monitoring
5 lakh	Product	200 support tickets overnight. A model update broke the output format. Legal wants to know what data you're storing. Only your co-founder knows the deploy password - and they're on leave.	SLA · queuing · legal · a team

Every rung adds a new category of problem. The question isn't whether it'll break - it's whether you'll know why.

Three questions to answer before climbing to the next rung.

1

Unit Economics

What does it cost per user per month at this scale?

2

Observability

When it breaks - who finds out first: you or your user?

3

Personalisation

Does the product feel personal to someone who isn't you?

These questions don't change. What changes is how much it costs you to get them wrong.

UNIT ECONOMICS

What does this actually cost?

ROI = Value per user − (Intelligence cost + Infrastructure cost + Hidden cost) per user per month

The model you pick is a business decision.

AI models aren't interchangeable. The cheapest model is 5–30× cheaper than the most expensive - and the math compounds fast at scale.

At 5 lakh users × 10 posts/month:

- Gemini Flash class: ~₹3–5 lakh/month
- GPT-4o class: ~₹15–50 lakh/month

The rule: use premium models for judgment tasks - decisions, nuance, tone. Use cheap models for mechanical tasks - templated generation, formatting, extraction.

Prices change. Always check the provider dashboard before you build.

Model	Approximate cost per post	Use it for
Gemini 2.5 Flash	~₹0.05–0.10	Mechanical generation
GPT-4o mini	~₹0.01–0.02	Light classification
GPT-4o / Claude Sonnet	~₹0.25–0.50+	Judgment and nuance

Estimates at ~400 input + 250 output tokens per post, Oct 2026. Verify at your provider's pricing page.

Free tiers are a starting line, not a business model.

How fast do I need to move? → Start managed (Supabase, Vercel) until speed costs more than money.

What does it cost at 10× my current users? → Run the math before you hit the ceiling.

Managed or raw cloud? → Stay managed until the bill exceeds the cost of someone to manage raw.

What's my lock-in tolerance? → Moving a database at scale is expensive. Know this before you start.

Layer	Free limit	You'll hit this at...	Paid tier
Frontend (Vercel)	Unlimited hobby	When you need SLA or team access	~₹1,700/month
Database (Supabase)	500MB, 50K rows	~500–1,000 active users with data	~₹2,100/month
Backend/server	Free at low traffic	Consistently above ~10K req/day	₹800–4,000/month
Domain	None	Day 1	~₹150/month

At 500 paying users, your infra bill is roughly ₹4,000–8,000/month. That's ₹8–16 per user. Factor it in before you set your price.

Your LinkedIn bot's unit economics.

The number that decides whether this is a hobby or a business.

Line	Per user
Revenue per user	₹499/month
− AI cost	₹1 (Gemini Flash, 10 posts)
− Infra per user	₹10 (₹4,750/month ÷ 500 users)
− Payment processing	₹13 (Razorpay, 2% + ₹3)
− Buffer (retries, support)	₹20
= Margin per user	**₹455 (~91%)**

91% gross margin. But this doesn't include: what it cost to acquire the customer, your time to maintain the build, or what happens when a model update breaks your prompts. Unit economics is the floor, not the ceiling.

Cutting cost is half the equation. The other half is value.

You've calculated your margin. But margin only stays healthy if users keep paying — and users only keep paying if the product keeps delivering.

A one-time build delivers value once. A reliable product compounds value over time — because it gets better as you learn what's working.

> The feedback loop is what turns a one-time build into a product. Without it, you're optimising blind.

What we would have tracked - and what it would have told us.

WHAT WE TRACKED AND HOW

Signal	Metric	How collected
Output	Publish rate	“Did you post this?” button after generation
Usage	Posts generated → published (funnel)	Session log per user
Retention	Day 7 and Day 14 active users	Login timestamp tracking
Qualitative	“What would make this better?”	One free-text field, optional

WHAT THE DATA SHOWED → WHAT WE BUILT

Publish rate: 34%

Qualitative feedback: “Too formal. Doesn’t sound like me.”

→ **Built:** Tone selector (Conversational / Professional / Storytelling) + voice onboarding (paste 3 of your own posts)

Day 14 retention: 38%

Usage funnel: most drop-off after generating 3 posts with no engagement

→ **Built:** Engagement tip after post 3 (“Your last post got X views — try this tweak”)

Result (30 days later):

Publish rate: 34% → 58% · Day 14 retention: 38% → 52%

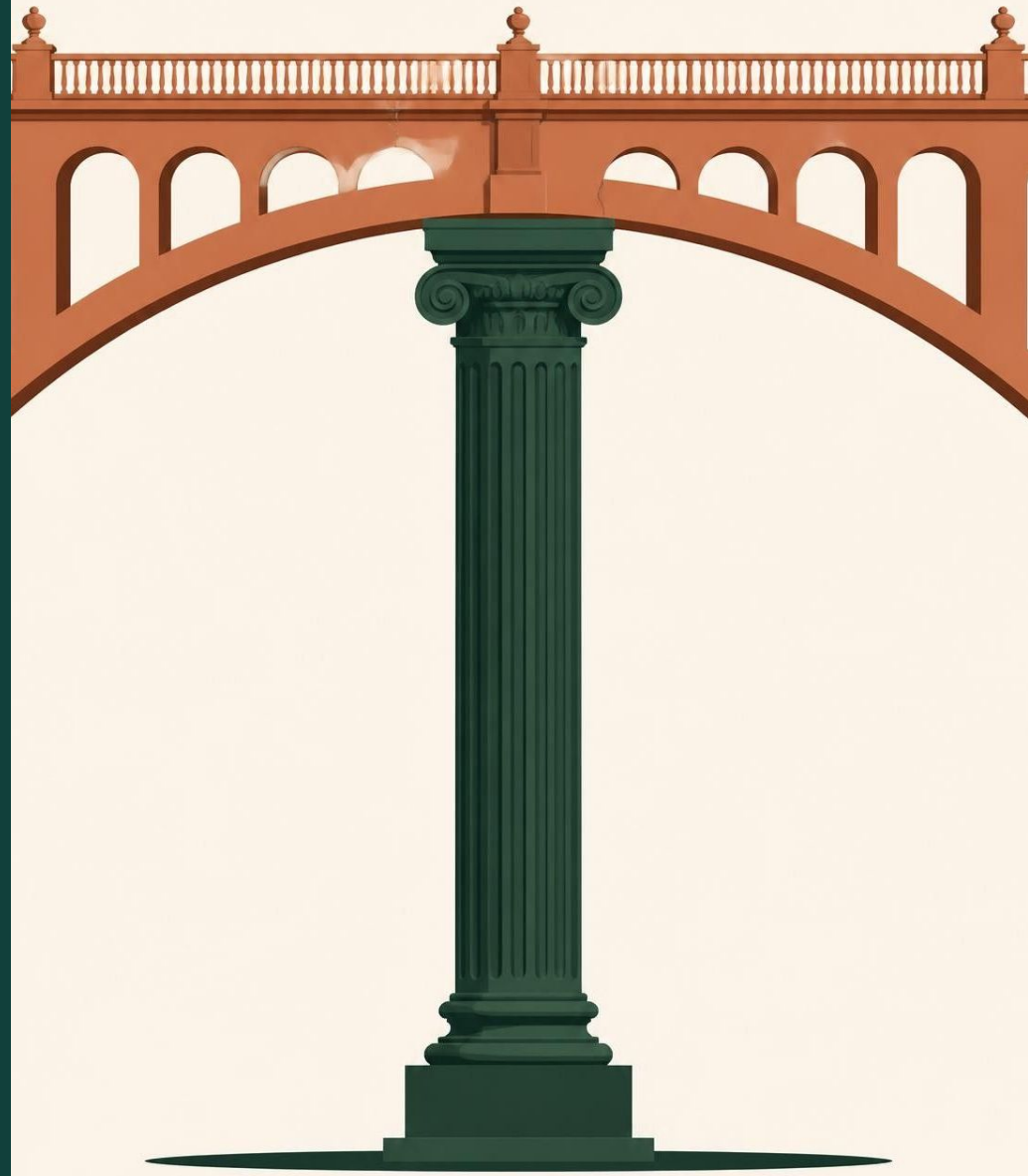
ⓘ This is the loop. Outcome defined → metric tracked → insight found → feature built → outcome improved. The build didn’t change. The product did.

DEPENDENCY TRAP

A build only you can operate is not a product. It's a personal dependency.

The shift from "I built something" to "I delivered a system" is exactly this:

Can someone else keep it running without you?



Most builds have three hidden dependencies on their builder.

DEPENDENCY 01

Operational dependency

Keys in your account. Deploy from your laptop. Only you can unpause the Supabase.

→ Visible. Fix first.

DEPENDENCY 02

Decision dependency

Edge cases require your judgment. Users message you when the build doesn't handle them.

→ Subtle. The product is using you as a fallback.

DEPENDENCY 03

Data dependency

No logs. No timestamps. No error context. When it breaks, Claude Code can read your code - but it can’t see what happened.

→ Most dangerous, because Claude Code can’t help you either.

Continuous improvement ≠ the product needing you to survive.

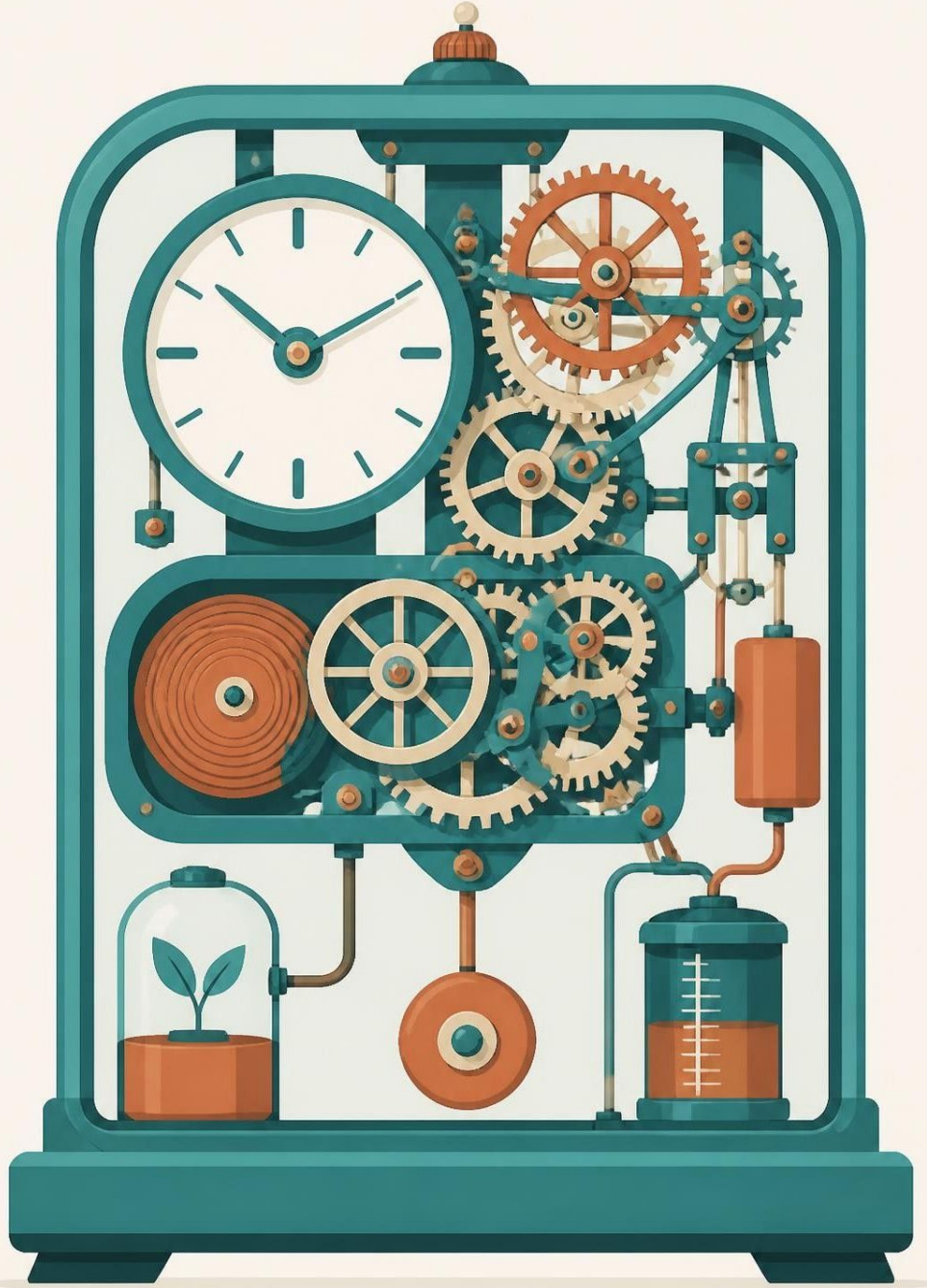
You will always be improving the product. That is not the same as the product requiring your presence to function.

The product that depends on you: if you step back for a month, users notice.

The product that doesn't: if you step back for a month, users keep getting value. When you return, you push improvements into a system that ran without you.

Continuous improvement is additive. It makes the product better.

It is not load-bearing. The product doesn't need your improvements to survive - it needs them to grow.



www.sherents.in

She–Rents

You built this. They shipped it.

India's first verified platform for women to find flats and flatmates.

Identity verified users. Safety check-in for every property visit. Compatibility-matched flatmates.

2,278 women in the community. Live on iOS and Android. Bangalore - and expanding.

One section of this cohort built something with the same core idea.

A flat-finding app — for a group of friends. That was your MRS.

The gap between what you built and what you're looking at is not talent.

It's not funding. It's the decisions we talked about today - made consistently, over time.



Five lessons. One product.

Everything from today lives inside sherents.in

Lesson	What it looks like in SheRents
Stack Diagnostic	Aadhaar verification is a Layer 5 dependency - external API. If onboarding breaks, that's where you start, not in the app code.
Scale	2,278 users puts them between Rung 1 and Rung 2. The next jump hits Supabase's free tier limit - infrastructure cost appears before they see it coming.
Unit Economics	AI cost per verified profile is fractions of a rupee. Margin comes from the verification moat — not from the AI.
Feedback Loop	Does a verified woman message another within 48 hours? That single funnel metric tells you if the product is working. Download count tells you nothing.
Independence	A woman in Bangalore can discover, verify, match, and move in - without the founder touching anything. That's the bar. Any step that requires manual intervention means the product isn't independent yet.

sherents.in · 2,278 verified users · Bangalore

> "SheRents started with a very real problem: house hunting as a woman is often unclear, uncomfortable, and sometimes unsafe. So, we built something better."

They didn't start with a pitch deck. They started with a problem they understood.

They built for themselves first. Then five people. Then two thousand.

You did the same thing this term.

The build is the easiest part.

The rest is what's to come in this course.

